

TECNOLOGÍA EN SISTEMAS Y GESTIÓN DE DATOS

Desarrollo de un Sistema Distribuido de Gestión de Productos utilizando SOAP, WSDL y Node.js

Informe técnico — Tarea práctica de Aplicaciones Distribuidas

Integrantes

Ismael [Apellido]

Alexandra [Apellido]

Israel [Apellido]

Asignatura: Aplicaciones Distribuidas

Docente: [Nombre del docente]

Julio de 2026

1. Introducción

Cuando una aplicación deja de vivir en una sola máquina y empieza a repartir su trabajo entre programas distintos que se comunican entre sí —estén en la misma computadora o en extremos opuestos del mundo—, hablamos de una aplicación distribuida. Esa arquitectura es, en la práctica, la forma en que funciona casi todo lo que usamos a diario: cuando alguien consulta el clima en su celular o revisa su banco en línea, hay un cliente pidiendo información y un servidor respondiéndola, casi siempre sin que el usuario note la separación.

Para que esa conversación entre cliente y servidor sea posible, ambos deben hablar el mismo idioma. SOAP (Simple Object Access Protocol) es uno de esos idiomas: un protocolo basado en XML que define reglas estrictas para empaquetar solicitudes y respuestas, apoyado en un documento llamado WSDL (Web Services Description Language) que describe, de forma exacta, qué operaciones ofrece un servicio, qué datos espera recibir y qué va a devolver. Es, en cierto sentido, un contrato: mientras el cliente y el servidor lo respeten, no importa en qué lenguaje de programación esté escrito cada uno.

Este informe documenta el desarrollo de un sistema de gestión de productos construido sobre ese modelo: un servidor SOAP hecho en Node.js, con su propio archivo WSDL, capaz de registrar, consultar, listar, actualizar, valorizar y eliminar productos. Para comprobar que el contrato funciona sin importar el lenguaje del cliente, lo consumimos desde dos clientes completamente distintos —uno en Python y otro en PHP— y, ya entendiendo bien el protocolo, decidimos ir un poco más allá de lo mínimo exigido: le agregamos persistencia en una base de datos (Supabase), una API REST sobre el mismo backend para comparar ambos protocolos en la práctica, y una interfaz web para cada cliente. Todo lo que se describe a continuación fue construido, ejecutado y probado por el equipo; no es un ejercicio únicamente teórico.

2. Objetivo general

Desarrollar una aplicación distribuida cliente-servidor que implemente un servicio web SOAP en Node.js, utilizando un documento WSDL para definir las operaciones, los parámetros de entrada y las respuestas del servicio de gestión de productos.

3. Objetivos específicos

- Comprender el funcionamiento del protocolo SOAP y la estructura de un documento WSDL, no solo en teoría sino construyendo uno desde cero.
- Implementar un servidor SOAP en Node.js que exponga las seis operaciones de gestión de productos exigidas por la tarea, con sus respectivas validaciones de negocio.
- Desarrollar clientes en dos lenguajes de programación distintos —Python y PHP— capaces de consumir el mismo servicio sin modificarlo.

- Intercambiar información estructurada mediante mensajes XML y comprobar, en la práctica, que el contrato definido en el WSDL se respeta en ambas direcciones.
- Probar cada una de las operaciones del servicio utilizando SoapUI, cubriendo tanto casos correctos como casos de error.
- Documentar todo el proceso de creación, ejecución y validación de la aplicación distribuida, incluyendo las decisiones técnicas tomadas en el camino.

4. Marco teórico

4.1 Aplicaciones distribuidas

Una aplicación distribuida es un sistema de software cuyos componentes se ejecutan en procesos separados —muchas veces en computadoras distintas— y que coordinan su trabajo enviándose mensajes a través de una red, en lugar de compartir memoria directamente. La ventaja de este enfoque es que cada componente puede desarrollarse, desplegarse y escalarse de forma independiente: en nuestro proyecto, por ejemplo, el servidor Node.js no necesita saber en qué lenguaje está escrito el cliente que lo consulta, y cada uno de los tres integrantes puede levantar su propia copia del servidor sin que eso afecte a los demás.

4.2 Servicios web

Un servicio web es una forma estandarizada de exponer funcionalidad de un sistema para que otros programas la consuman a través de la red, generalmente sobre HTTP. En nuestro caso, el mismo servidor Node.js expone dos servicios web distintos sobre el mismo puerto: uno bajo el protocolo SOAP (en la ruta /productos) y, como mejora adicional, otro bajo el estilo REST (en las rutas /api/*). Ambos terminan ejecutando exactamente la misma lógica de negocio, lo cual nos permitió comparar en la práctica cómo cada protocolo empaqueta y transporta la misma información.

4.3 Protocolo SOAP

SOAP es un protocolo de mensajería basado en XML que define un formato estricto para el intercambio de información entre aplicaciones: cada mensaje viaja dentro de un "sobre" (Envelope) que contiene un encabezado opcional y un cuerpo (Body) con la solicitud o la respuesta. A diferencia de otros estilos de comunicación más flexibles, SOAP exige que la estructura de cada mensaje coincida exactamente con lo que el WSDL declaró; si el servidor incluye un dato que el contrato no definió, el mensaje deja de ser válido. Esto lo aprendimos de la peor manera: al agregar un campo adicional para identificar qué integrante había registrado cada producto, la librería que usamos terminó incluyéndolo también en las respuestas SOAP, rompiendo la conformidad del contrato. Tuvimos que separar explícitamente esa información para que solo viajara por la API REST, y dejar el SOAP tal como el WSDL lo describe.

4.4 Lenguaje XML

XML (Extensible Markup Language) es un lenguaje de marcado diseñado para representar información estructurada de forma legible tanto para personas como para programas, usando etiquetas que el propio autor del documento define según lo que necesite describir. SOAP se apoya completamente en XML: cada solicitud que enviamos desde SoapUI —por ejemplo, para registrar un producto— y cada respuesta que el servidor devuelve, es un documento XML válido, con las etiquetas <codigo>, <nombre>, <precio>, etc., anidadas dentro del sobre SOAP.

4.5 Archivo WSDL

El WSDL es el documento que describe formalmente un servicio SOAP: qué operaciones ofrece, qué tipos de datos usa cada una (types), qué mensajes de entrada y salida corresponden a cada operación (message y portType), cómo se transportan esos mensajes (binding) y en qué dirección de red se puede acceder al servicio (service y port). En este proyecto escribimos el archivo productos.wsdl a mano, definiendo las seis operaciones exigidas por el enunciado, y lo dejamos accesible desde el navegador en <http://localhost:8000/productos?wsdl>, tal como pide la tarea. Ese mismo archivo es el que SoapUI necesita para generar automáticamente las solicitudes de prueba, y el que las librerías cliente (zeep en Python, SoapClient en PHP) leen para saber qué operaciones pueden invocar.

4.6 Arquitectura cliente-servidor

La arquitectura cliente-servidor divide un sistema en dos roles: el servidor, que concentra los datos y la lógica de negocio y espera solicitudes, y el cliente, que las envía y presenta los resultados. En nuestro proyecto el servidor es único (Node.js, con su lista de productos en memoria), pero tiene múltiples clientes que no se conocen entre sí: dos clientes de consola (Python y PHP) que demuestran las operaciones exigidas por la tarea, y dos paneles web —uno por cada lenguaje— contruidos como mejora adicional. Ninguno de los clientes necesita saber cómo está implementado el servidor por dentro; les basta con el contrato que describe el WSDL.

4.7 SOAP frente a REST

REST (Representational State Transfer) es un estilo arquitectónico —no un protocolo con una especificación única como SOAP— que aprovecha directamente los verbos de HTTP (GET, POST, PATCH, DELETE) para representar acciones sobre recursos, normalmente intercambiando datos en JSON. SOAP, en cambio, siempre usa XML, define su propio contrato formal (el WSDL) y es más rígido en cuanto a validación y tipado. Al construir ambas versiones sobre el mismo backend —SOAP en /productos y REST en /api/productos, compartiendo la misma lógica de validación en operaciones.js— pudimos comparar directamente: la versión REST resultó más liviana de invocar (una simple petición HTTP con JSON) y más fácil de consumir desde herramientas como Postman, mientras que la versión SOAP, aunque más verbosa,

ofrece un contrato explícito y autodescriptivo que SoapUI puede leer sin que nosotros documentemos nada aparte.

5. Arquitectura de la aplicación

El diagrama siguiente resume cómo está organizado el sistema tal como quedó al final del desarrollo. Cada integrante del equipo corre su propia instancia local del servidor Node.js (puerto 8000), que expone el mismo contrato SOAP y la misma API REST; las tres instancias comparten una única base de datos en Supabase, que actúa como persistencia adicional sobre la lista en memoria de cada servidor.

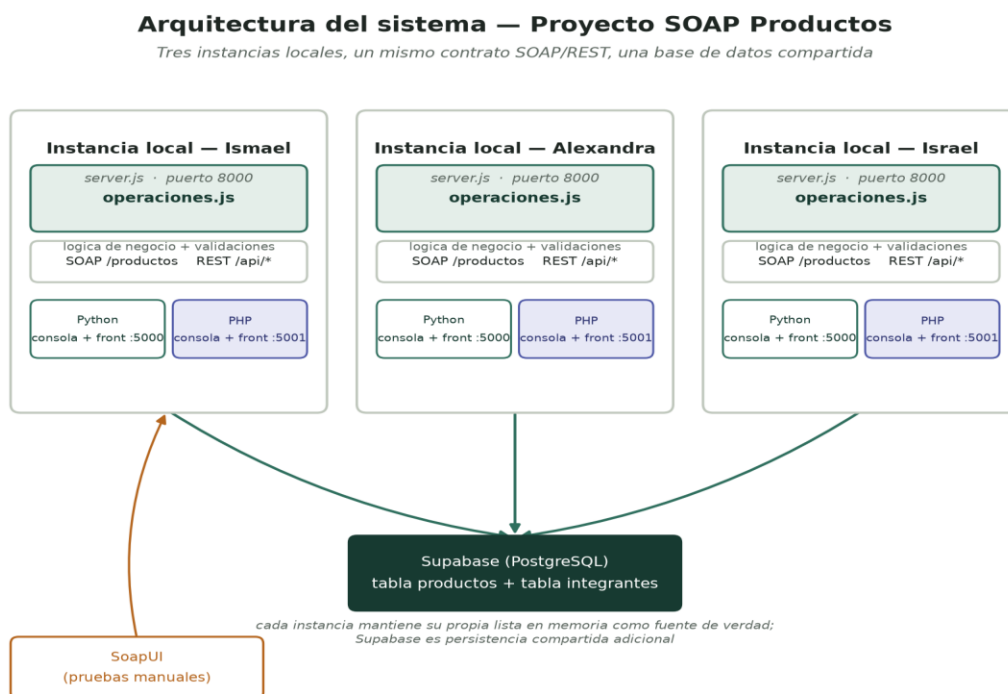


Figura 1. Arquitectura general del sistema: tres instancias locales, un mismo contrato SOAP/REST, una base de datos compartida.

La pieza central del servidor es `operaciones.js`: un módulo que concentra toda la lógica de negocio y las validaciones de las seis operaciones. Tanto el servicio SOAP (`server.js`) como la API REST (`rest.js`) son, en el fondo, envoltorios delegados sobre ese mismo módulo — así garantizamos que, sin importar por cuál de los dos protocolos entre una solicitud, la respuesta ante los mismos datos sea idéntica. Por debajo, `data/productos.js` mantiene el arreglo en memoria que es la fuente de verdad operativa (tal como exige el enunciado), mientras que `db/supabaseClient.js` intenta reflejar cada cambio en Supabase de forma adicional; si Supabase no responde, el servidor sigue funcionando exclusivamente en memoria, sin caerse.

Sobre ese mismo servidor se apoyan cuatro clientes: dos de consola (`cliente-python/client.py` con la librería `zeep`, y `cliente-php/client.php` con la clase nativa `SoapClient`) que son los exigidos por el

enunciado, y dos paneles web —uno por cada lenguaje— que construimos como mejora adicional para poder operar el servicio desde el navegador en lugar de la terminal.

6. Desarrollo

6.1 Instalación de Node.js y creación del proyecto

El punto de partida fue instalar Node.js y crear la carpeta del proyecto siguiendo la estructura sugerida por el enunciado (servidor/, cliente-python/, cliente-php/, evidencias/, informe/). Dentro de servidor/ inicializamos el proyecto con npm y agregamos las dependencias necesarias:

```
npm install express soap
```

6.2 Diseño del archivo WSDL

Antes de escribir una sola línea del servidor, definimos el contrato: creamos productos.wsdl declarando el tipo complejo Producto (código, nombre, categoría, precio, cantidad) y, para cada una de las seis operaciones, su mensaje de entrada, su mensaje de salida, la entrada en el portType, y su asociación al binding SOAP sobre HTTP. Al probarlo por primera vez con node-soap nos topamos con un error de "comentario mal formado": habíamos usado líneas decorativas con guiones dobles (--) dentro de comentarios XML, algo que la especificación XML prohíbe explícitamente. Corregirlo nos obligó a entender que un WSDL, al final, es un documento XML como cualquier otro y hereda todas sus reglas.

6.3 Implementación del servidor

Con el contrato definido, implementamos server.js utilizando la librería soap sobre un servidor Express, montando el servicio en la ruta /productos y dejando el WSDL disponible en /productos?wsdl. Cada una de las seis operaciones quedó implementada validando primero los datos de entrada (código no vacío, no duplicado, nombre y categoría no vacíos, precio mayor que cero, cantidad entera igual o mayor que cero, y existencia del producto cuando aplica), y devolviendo siempre un objeto con los campos estado y mensaje, tal como pide el enunciado. Cada operación ejecutada queda registrada en la consola del servidor con hora y resultado, lo que facilitó muchísimo depurar errores durante las pruebas.

6.4 Persistencia adicional con Supabase

Como mejora adicional, conectamos el servidor a una base de datos Postgres administrada por Supabase. El diseño es híbrido a propósito: el arreglo en memoria sigue siendo la fuente de verdad para responder cada solicitud (así el servidor jamás depende de que Supabase esté disponible), pero cada operación que modifica datos también intenta reflejar el cambio en la base compartida. Como los tres integrantes corremos nuestra propia instancia local apuntando a la misma base, agregamos una columna origen que registra qué instancia creó cada producto, y un endpoint adicional (GET /api/instancia) que cada panel web consulta para mostrar con qué servidor está hablando en ese momento.

6.5 Implementación de los clientes

Desarrollamos dos clientes de consola, uno por integrante asignado: `client.py` en Python, usando la librería `zeep`, que lee el WSDL y genera dinámicamente las funciones correspondientes a cada operación; y `client.php` en PHP, usando la clase nativa `SoapClient`, sin ninguna librería externa. Ambos ejecutan la misma secuencia de pruebas: registran dos productos, consultan uno existente y uno inexistente, listan el inventario, actualizan el stock de un producto, calculan el valor de su inventario y finalmente lo eliminan —incluyendo, en cada paso relevante, un intento con datos inválidos para comprobar que el servidor rechaza correctamente el caso incorrecto.

6.6 API REST y paneles web (mejora adicional)

Para poner en práctica la comparación entre SOAP y REST, expusimos las mismas seis operaciones también como una API REST (GET, POST, PATCH y DELETE sobre `/api/productos`), reutilizando la misma lógica de validación del servidor SOAP. Sobre esa base construimos dos paneles web —uno en Flask para Python, otro en PHP puro sin framework— con un menú lateral que organiza las seis operaciones en el mismo orden del enunciado, cada uno con su propio color de acento para distinguirlos a simple vista, y ambos mostrando en tiempo real con qué instancia del servidor están hablando y qué integrante registró cada producto.

6.7 Pruebas

Antes de dar por cerrado el desarrollo, importamos `productos.wsdl` en SoapUI y ejecutamos al menos un caso correcto y un caso incorrecto por cada una de las seis operaciones (trece casos en total, incluyendo un caso estructural de SOAP Fault sobre `ListarProductos`, que no admite parámetros y por tanto no tiene un caso incorrecto de negocio propio). Las capturas de estas pruebas se documentan en la sección de evidencias.

7. Evidencias

Esta sección reúne las capturas que demuestran el funcionamiento real del sistema. El conjunto completo de evidencias de SoapUI (14 archivos) está disponible en la carpeta `evidencias/` del repositorio; aquí se incluye una muestra representativa.

7.1 WSDL importado en SoapUI

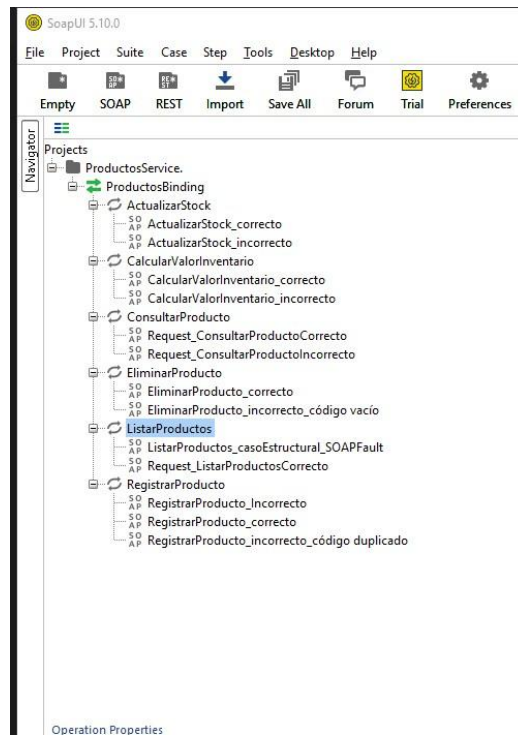


Figura 2. Árbol de operaciones generado por SoapUI al importar productos.wsdl — las seis operaciones quedan disponibles, cada una con su caso correcto e incorrecto.

7.2 Caso correcto — RegistrarProducto

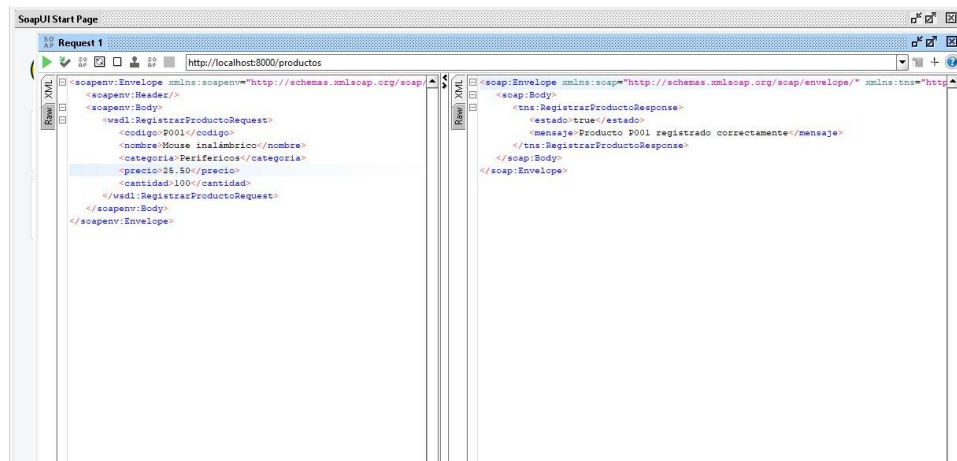


Figura 3. Solicitud y respuesta SOAP de un registro exitoso (estado = true).

7.3 Caso incorrecto — código duplicado

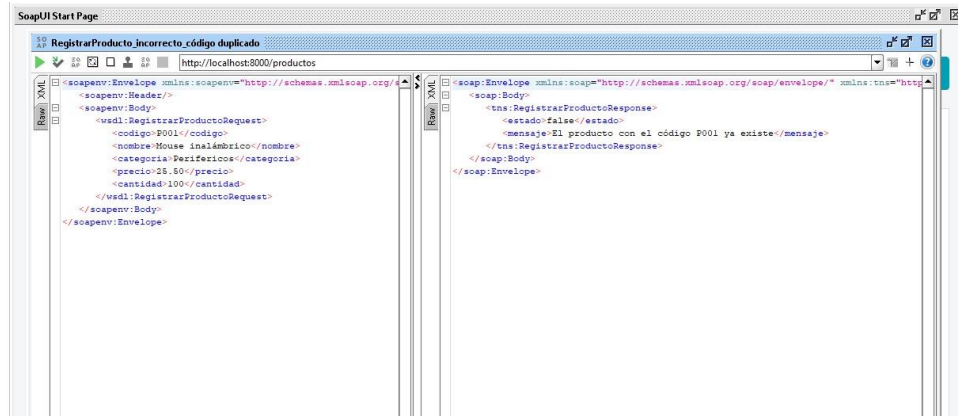


Figura 4. El servidor rechaza un código ya existente, devolviendo estado = false y un mensaje explicando el motivo.

7.4 Caso incorrecto — producto inexistente

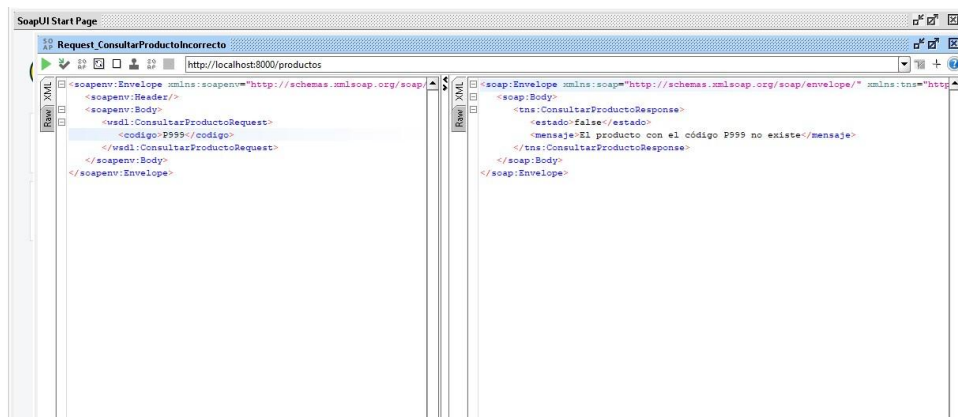


Figura 5. ConsultarProducto ante un código que no existe.

7.5 Caso estructural — SOAP Fault

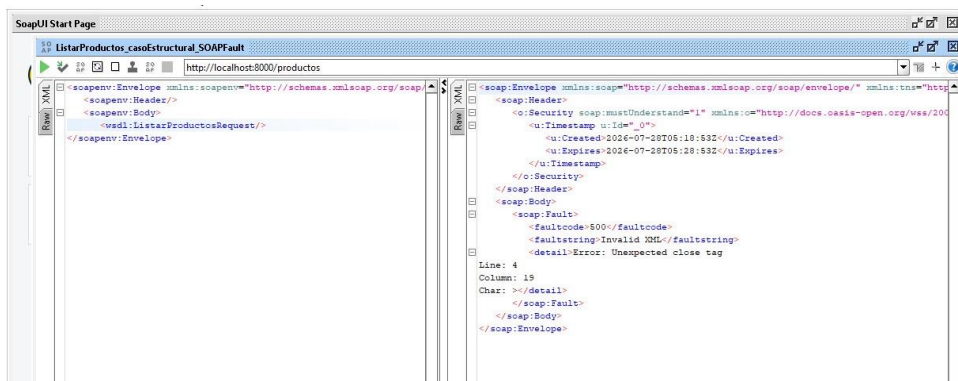


Figura 6. Como ListarProductos no recibe parámetros, se documentó en su lugar un XML mal formado para mostrar cómo el motor SOAP responde con un Fault ante una solicitud inválida en su estructura.

7.6 Caso correcto — EliminarProducto

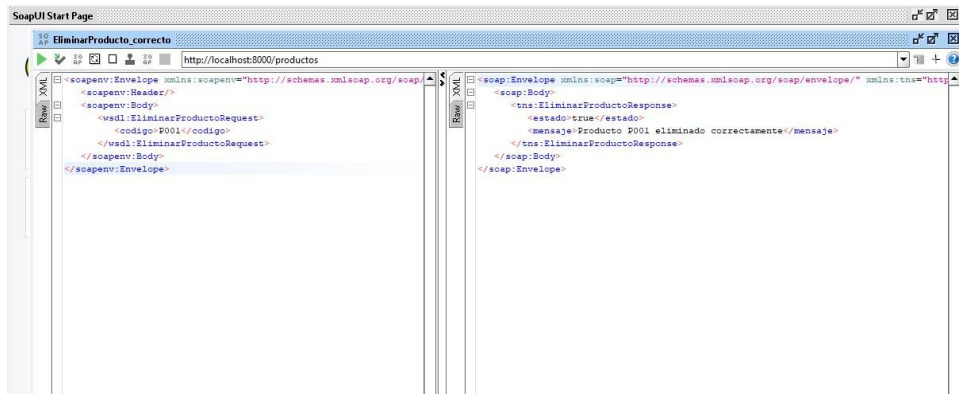


Figura 7. Eliminación exitosa de un producto.

7.7 Ejecución del servidor

El servidor se levanta con `npm start` desde la carpeta `servidor/` y muestra en consola la hidratación desde Supabase, la dirección del servicio SOAP y de la API REST, y cada operación ejecutada con su resultado.

[ESPACIO PARA CAPTURA] Terminal con el servidor corriendo

Ejecutar `"npm start"` dentro de `servidor/` y capturar la consola mostrando las líneas `"Servidor SOAP escuchando..."`, `"WSDL disponible..."` y `"API REST disponible..."`.

7.8 Acceso al WSDL desde el navegador

[ESPACIO PARA CAPTURA] Navegador mostrando el WSDL

Abrir `http://localhost:8000/productos?wsdl` en el navegador (con el servidor corriendo) y capturar el XML completo que se muestra.

7.9 Ejecución de los clientes

Con el servidor corriendo, cada cliente se ejecuta de forma independiente y muestra en su propia consola el resultado de las 7 demostraciones mínimas (registro, consulta existente/inexistente, listado, actualización de stock, cálculo de inventario y eliminación).

[ESPACIO PARA CAPTURA] Consola del cliente Python

Ejecutar `"python client.py"` dentro de `cliente-python/` y capturar la salida completa en la terminal.

[ESPACIO PARA CAPTURA] Consola del cliente PHP

Ejecutar `"php client.php"` dentro de `cliente-php/` y capturar la salida completa en la terminal.

8. Análisis de resultados

El servicio SOAP respondió de forma consistente a lo largo de todas las pruebas: las seis operaciones funcionan tanto desde SoapUI como desde los dos clientes de consola y los dos paneles web, y las validaciones de negocio se comportan igual sin importar por cuál de las dos vías —SOAP o REST— llegue la solicitud, gracias a que ambas comparten la misma capa de lógica (operaciones.js). Esto nos permitió confirmar en la práctica algo que solo habíamos leído en la teoría: un buen contrato de servicio (ya sea un WSDL o una API REST bien diseñada) permite que clientes completamente distintos —de consola o web, en Python o en PHP— interactúen con el mismo backend sin conocer los detalles de su implementación.

La comunicación cliente-servidor se dio siempre sobre HTTP, con el cuerpo de cada mensaje en XML para el canal SOAP y en JSON para el canal REST. El único inconveniente real que enfrentamos fue el ya mencionado en el marco teórico: al agregar el campo origen para identificar la instancia que registraba cada producto, la librería node-soap lo serializó también en las respuestas SOAP a pesar de no estar declarado en el WSDL, lo que rompía la conformidad estricta del contrato. Detectarlo nos obligó a inspeccionar el XML crudo de la respuesta —no solo la versión ya interpretada por el cliente— y a separar explícitamente qué datos viajan por cada protocolo.

Trabajar con tres integrantes corriendo cada uno su propio servidor local, apuntando a la misma base de datos compartida en Supabase, también puso a prueba un supuesto que dábamos por sentado: que "nuestros datos" eran solo nuestros. Más de una vez, al listar productos, aparecían registros de prueba de otro integrante. Resolverlo (con la columna origen y el identificador de instancia) terminó siendo, sin haberlo planeado, una lección adicional sobre los problemas reales de compartir estado entre sistemas distribuidos.

9. Conclusiones

- SOAP resultó ser, tal como plantea la teoría, un protocolo más rígido y verboso que REST, pero esa misma rigidez es la que permite que herramientas como SoapUI generen automáticamente solicitudes de prueba a partir del WSDL, sin que el equipo tenga que documentar manualmente cada operación aparte.
- Un WSDL bien definido es, en la práctica, más que documentación: es el contrato que hace posible que clientes en Python y PHP —que no comparten ni una línea de código entre sí— consuman exactamente el mismo servicio sin errores de interpretación.
- Separar la lógica de negocio (operaciones.js) de la capa de transporte (SOAP en server.js, REST en rest.js) resultó clave para mantener ambos protocolos consistentes entre sí, y facilitó detectar y corregir el problema de serialización que describimos en el análisis de resultados.
- Diseñar el servidor para que la persistencia en base de datos sea un complemento y no una dependencia (memoria como fuente de verdad, Supabase como mejora adicional) demostró ser

una decisión acertada: nunca perdimos funcionalidad por una caída o mala configuración de la base de datos.

- Ir más allá del mínimo exigido —agregando la API REST y los paneles web— terminó siendo la parte que más nos ayudó a entender las diferencias reales entre SOAP y REST, más que cualquier lectura teórica previa.

10. Recomendaciones

- Antes de agregar cualquier campo nuevo a una respuesta SOAP, revisar el XML crudo generado (no solo la versión ya parseada por el cliente) para confirmar que sigue siendo válido contra el WSDL declarado.
- Si un equipo va a compartir una misma base de datos entre varias instancias locales del mismo servidor, definir desde el inicio algún identificador de origen — evita confusión y facilita depurar cuando los datos de prueba de distintos integrantes se mezclan.
- Para proyectos futuros similares, valdría la pena agregar autenticación básica a los paneles web antes de considerarlos listos para un entorno que no sea de desarrollo local, ya que en esta versión cualquiera con acceso a la red puede registrar o eliminar productos sin restricción.

11. Bibliografía

Bray, T., Paoli, J., Sperberg-McQueen, C. M., Maler, E., & Yergeau, F. (2008). Extensible Markup Language (XML) 1.0 (5.ª ed.). World Wide Web Consortium. <https://www.w3.org/TR/xml/>

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). Distributed systems: Concepts and design (5.ª ed.). Addison-Wesley.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

OpenJS Foundation. (2024). Node.js documentation. <https://nodejs.org/en/docs>

Tanenbaum, A. S., & Van Steen, M. (2017). Distributed systems: Principles and paradigms (3.ª ed.). Pearson.

World Wide Web Consortium. (2001). Web Services Description Language (WSDL) 1.1. <https://www.w3.org/TR/wsdl>

World Wide Web Consortium. (2007). SOAP Version 1.2 Part 0: Primer (2.ª ed.). <https://www.w3.org/TR/soap12-part0/>